

DBMS课程设计

Compile sql statements to
design databases

第11组

单楚雄 林裕涛 万兴海 陈正江 赵奕森

01

小组成员与分工

Team members and
division of labor

02

项目设计

Project design

03

项目功能实现

Project function
realization

04

总结与体会

Summary and
experience

小组成员及分工

Team members and division of labor

Each team member is responsible for implementing the code and reporting on it



01

单楚雄

主要负责前端设计，包括用户界面的设计和用户交互的优化。

负责任务分工，确保每个成员明确自己的任务和责任。

负责文档的汇总，包括项目计划、设计文档、进展报告等。

美化PPT，制作、美化项目演示和汇报的PPT文档。



02

林裕涛

主要负责系统框架设计，设计整个系统的结构和架构。

实现语法分析，确保系统能够正确解析用户输入的SQL语句。

负责全局代码优化，提高系统的性能和效率。



03

陈正江

主要负责工具类Utils包的实现，编写通用的工具函数。

PPT主要制作，提供项目中工具类的文档资料。

参与接口的实现与调用，确保不同模块之间的协作。



04

赵奕森

主要负责语句处理部分，编写处理SQL语句的代码。

PPT主要制作，准备演示和汇报的内容。

与其他成员协作，确保系统的各个部分能够正确协同工作。



05

万兴海

主要负责系统需求分析，与客户或团队成员沟通，明确系统的需求和功能。

实现管理表和日志CREATE包，确保数据库的管理和记录操作。

提供项目需求和功能的文档资料。

项目设计

1、需求分析

实现功能：

- ①图形化GUI；
- ②系统可运行；
- ③复现磁盘操作、缓冲区管理内容；
- ④能建立数据库数据表；
- ⑤能插入多种类型的数据；
- ⑥能实现增删改查CRUD；
- ⑦登录功能（分管理员和非管理员，拥有权限不同）；
- ⑧日志功能（每个库维护一个日志，每次使用一个库会重新创建一个日志，日志命名为：当前时间+UUID后段）。

性能需求：

- ①满足并发条件下的数据交互需求；
- ②达到可以较为迅速的处理SQL语句和较短的数据返回时间；
- ③能够完成适当吞吐量的数据存取。

安全性：

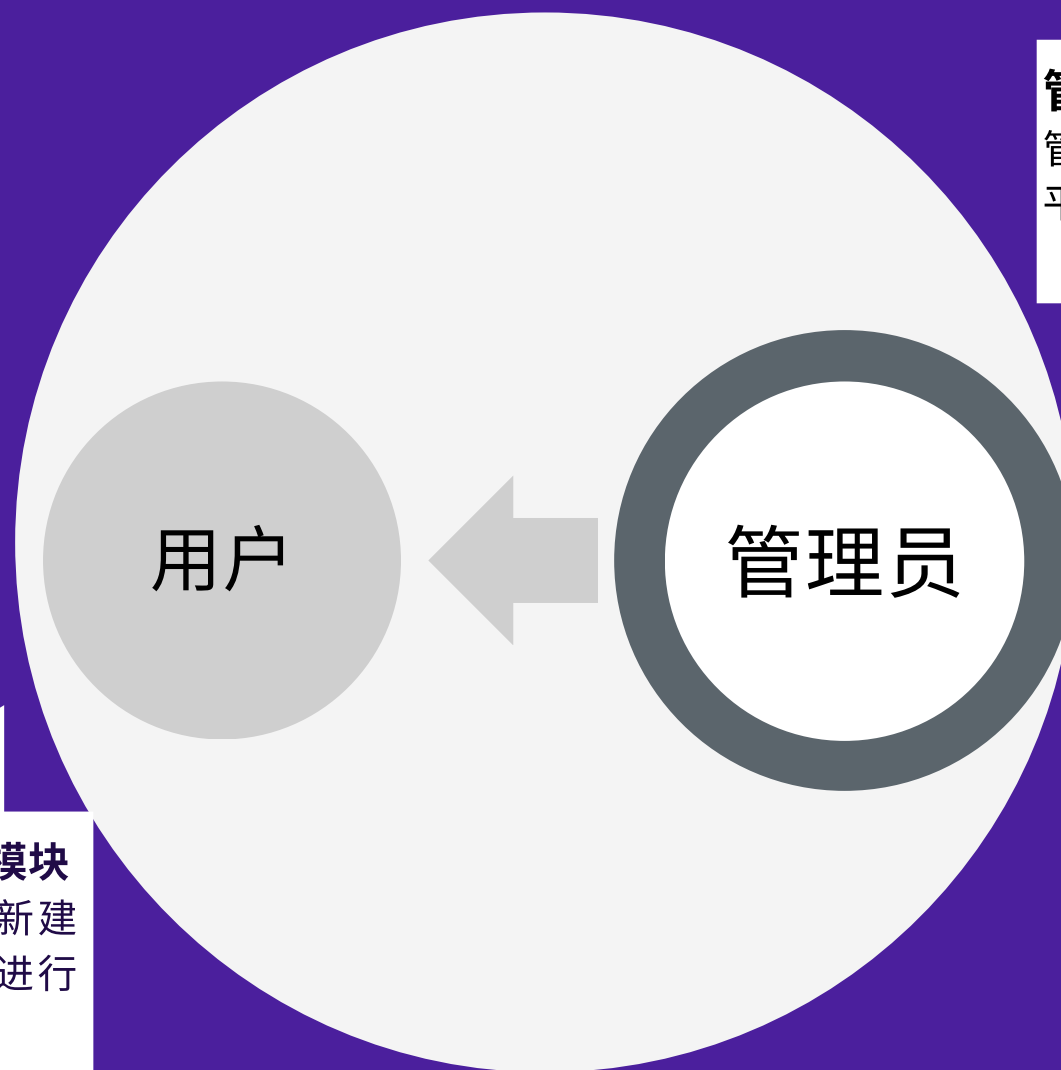
存储在本地磁盘文件，安全性高；
系统操作可复现。

项目设计

2、架构设计

用Java实现DBMS-lite数据库管理系统；
用Swing构件实现图形界面GUI；
使用java的RandomAccessFile类实现文件读写部分。

用户个人信息管理模块
用户登录、注册、新建库、新建表、对表进行增删改查等功能。



管理员信息管理模块
管理员负责“DBMS-lite”平台的日常管理。

项目设计

3、数据库设计：

语法分析部分

- 用于处理数据类型的Datatype类
- 用于语法分析,处理SQL语句类型的StatementType类和处理where子句的WhereType类

sql语句处理部分

- 处理Create语句的CreateStatement类，可以使用库（同时创建日志）、创建库、创建表，创建并写入日志
- 按照where语句条件删除表中信息的DeleteStatement类，以及处理对应插入语句的InsertStatement类、选择语句的SelectStatement类、按照条件更新表某行某列数据的UpdateStatement类

工具类——提供一些常见的操作、处理和工具，使得代码更加模块化、可维护和可重用。

- Page表示内存中页面，使用ByteBuffer类来申请空间
- SchemaInfo与SchemaFile交互，读取SchemaFile文件，可进一步获取对应表的表结构信息 打开、读取、关闭。
- User用来表示和管理用户的信息
- FileMgr负责管理当前打开的数据库下的表或日志

项目设计

4、界面设计：

1.便捷使用者操作，突出关键内容

用户界面内容需要包括有用户对象名称，欢迎信息、输入的文本框、显示结果的文本区域的组件。

2.实现事件监听和内容交互

主要实现对文本框实现事件监听，按下回车键后会处理文本框内的内容。调用 StatementType 类分析执行 SQL 语句，处理执行结果，并将结果显示在文本区域中。

项目功能实现

IN TERMS OF PROJECT FUNCTIONAL I

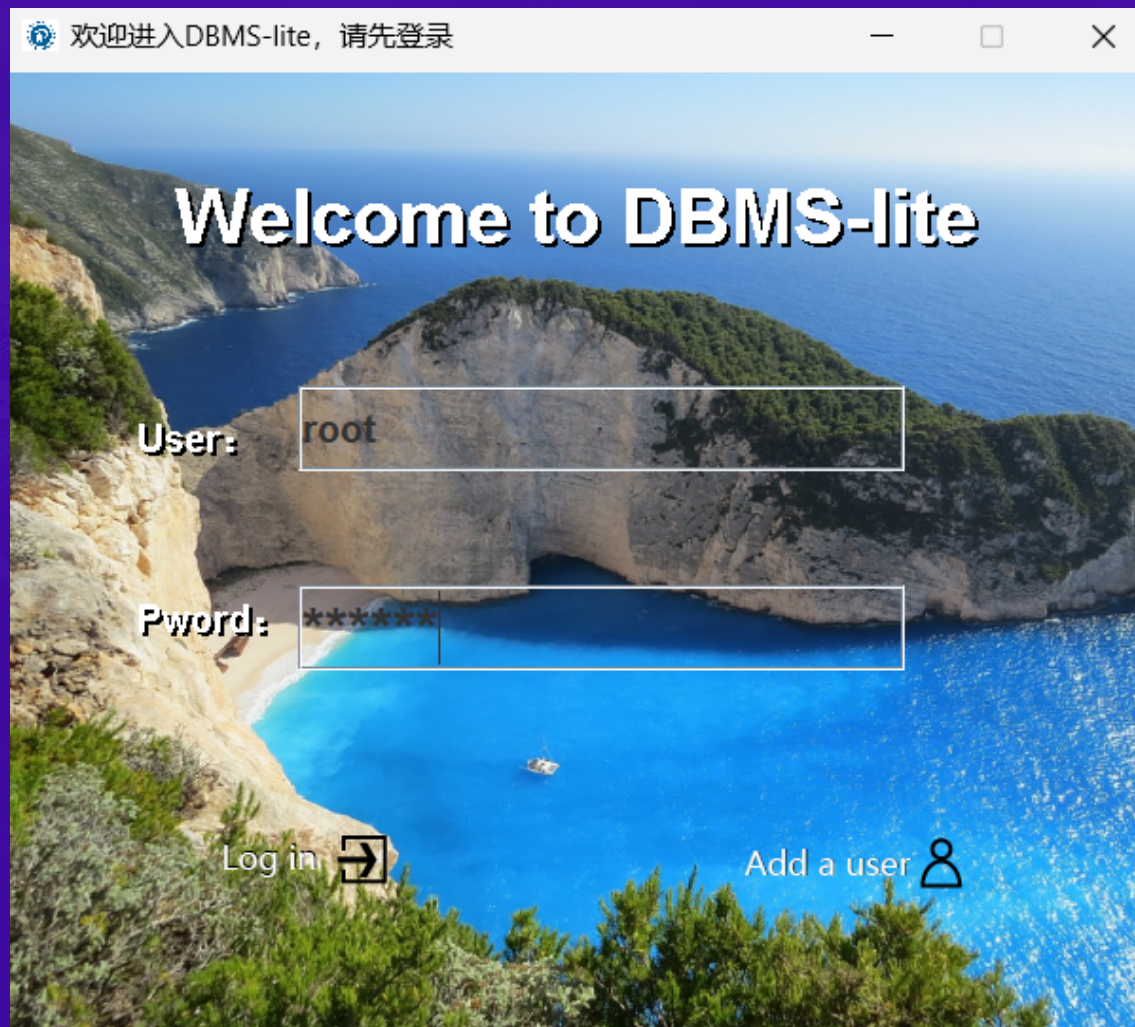
CODE IMPLEMENTATION METHOD,LANGUAGE, IMPLEMENTATION,UNIT TESTING AND
INTEGRATION TESTING,VERSION CONTROL

01 用户界面登录

根据对应的用户名以及密码来判断是否为管理员或者普通用户

Login.java：用于用户登录的Java Swing界面的类，Login类继承Jframe（顶层容器），表示登录界面的窗口。

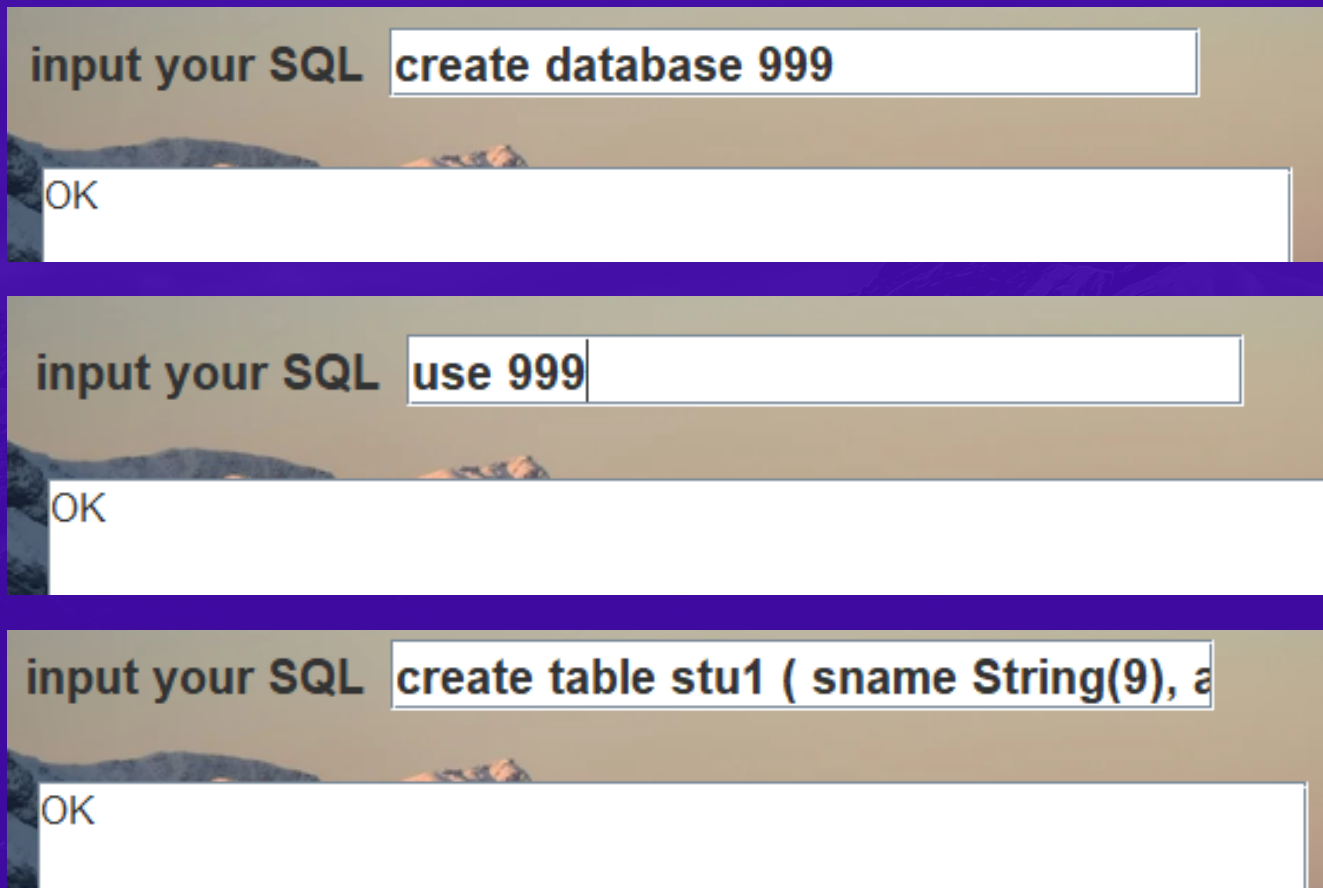
Mainview.java：主要对文本框实现事件监听，按下回车键后会处理文本框内的内容。



02 数据库/表的建立

CreateStatement.java: 根据输入SQL语句来执行相应的CREATE操作。

- 以use开头则进入指定数据库，不存在则报错
- 以create database开头则创建新的数据库目录
- 以create table开头则创建新的数据表文件，同时写入模式表schema.dbs
- 其中利用getNameAndAttr方法表提取表结构信息并返回二维数组



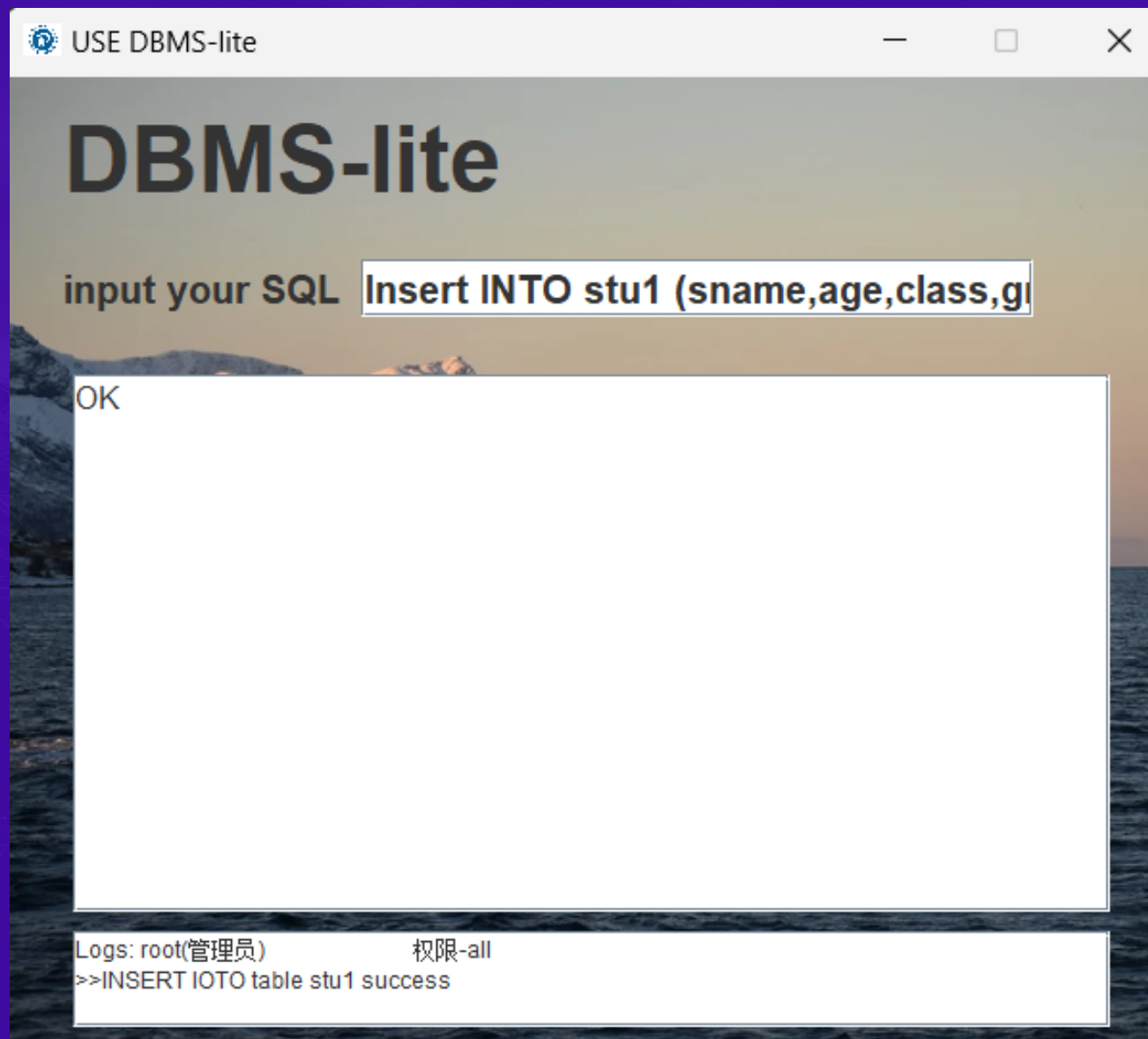
其中在一个数据库中可建立多张表

03 数据插入

InsertStatement.java:
processInsert方法处理
insert语句，一次只能插
入一条。无数据则置为0
或""。插入完成后关闭
RandomAccessFile。

没有赋值的设为默认值

- 整形默认值0
- double默认0.0
- 字符串默认""



04 数据查询

SelectStatement.java:

- 1.查询语句解析：将where和from分离，获取查询条件和表名。
- 2.条件处理：通过 WhereType 类的 getType 方法获取满足条件的行号。
- 3.字段处理：根据查询语句中的字段部分，获取要查询的字段名。

其中where语句处理：

- 支持等值比较、不等值比较、大于、小于等条件。
- 如果条件中包含字符串（' '），去除字符串引号后再进行等值比较。

input your SQL `select * from stu1`

flag	sname	age	class	grade
true	张11	16	2019	95.0
true	张12	0	2019	85.0
true	张13	17	2019	0.0
true	张14	18		65.0
true	张15	15	2020	65.0

input your SQL `select * from stu1 where age < 18;`

flag	sname	age	class	grade
true	张11	16	2019	95.0
true	张12	0	2019	85.0
true	张13	17	2019	0.0
true	张15	15	2020	65.0

input your SQL `grade from stu1 where class = '2019';`

sname	age	grade
张11	16	95.0
张12	0	85.0
张13	17	0.0

05 数据的删除和更新

DeleteStatement.java :
按照where语句条件删除表中信息。
本质上是将表中指定行的标志位flag置为false。
其中管理员可以看到flag为false的信息，普通用户则看不到。

input your SQL

OK

input your SQL

flag	sname	age	class	grade
false	张11	16	2019	95.0
true	张12	0	2019	85.0
true	张13	17	2019	0.0
true	张14	18		65.0
true	张15	15	2020	65.0

input your SQL

OK

input your SQL

flag	sname	age	class	grade
false	张11	16	2000	95.0
true	张12	0	2000	85.0
true	张13	17	2019	0.0
true	张14	18		65.0
true	张15	15	2020	65.0

UpdateStatement.java :
按照条件来更新表某行某列数据。

06 日志功能

每个数据库维护一个日志，每次使用一个库回重新创建一个日志

记录日志信息：

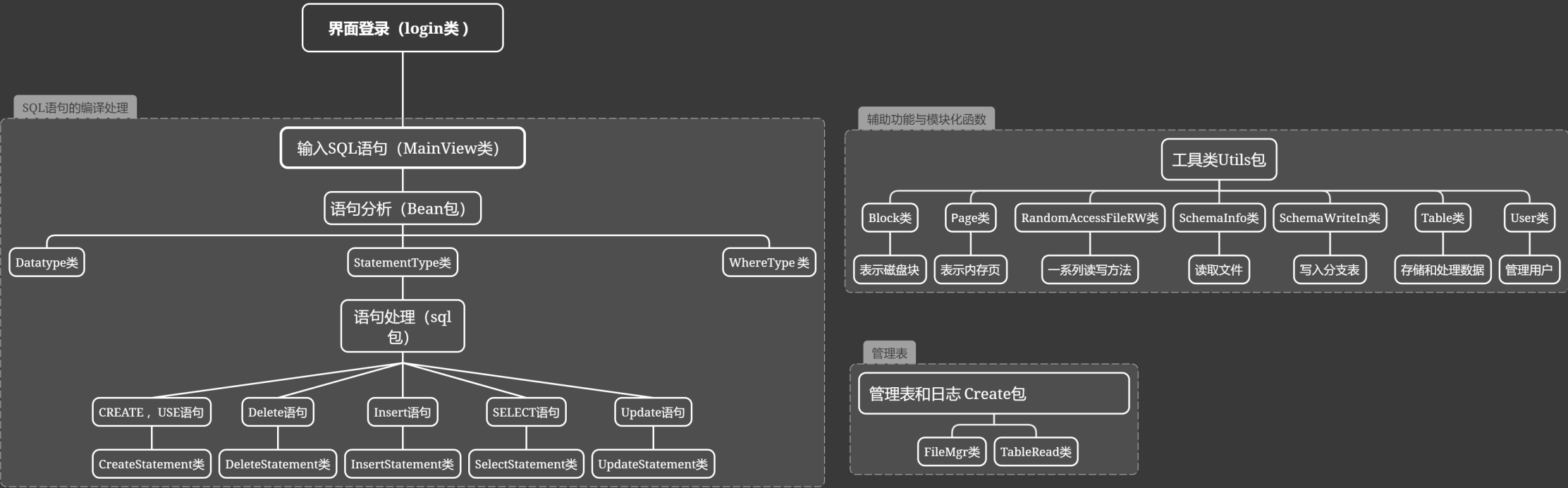
```
int startpos = 0;
int npos = startpos + Page.maxLength(df1.format(day).length());
byte[] b = new byte[npos * 2]; // 记录的总长度，4字节是整数长度
FileMgr fLog = new FileMgr( fileName: logPath+"\\ "+LogName, blkSize:
// 字节数组b,是内存页的实际存储空间
Page p = new Page(b); // 按照记录长度（即，字节数组）建立内存页
Block blk = new Block( filename: "\\logs"+"\\ "+LogName, blockID: 0);
p.setString(startpos, df1.format(day)); // 把该条记录的字符串按位置sta
fLog.write(blk, p);
p.setString(npos, s: "use this database");
fLog.write(blk, p);
```

命名格式: "yyyyMMdd" + Uuid + ".log"

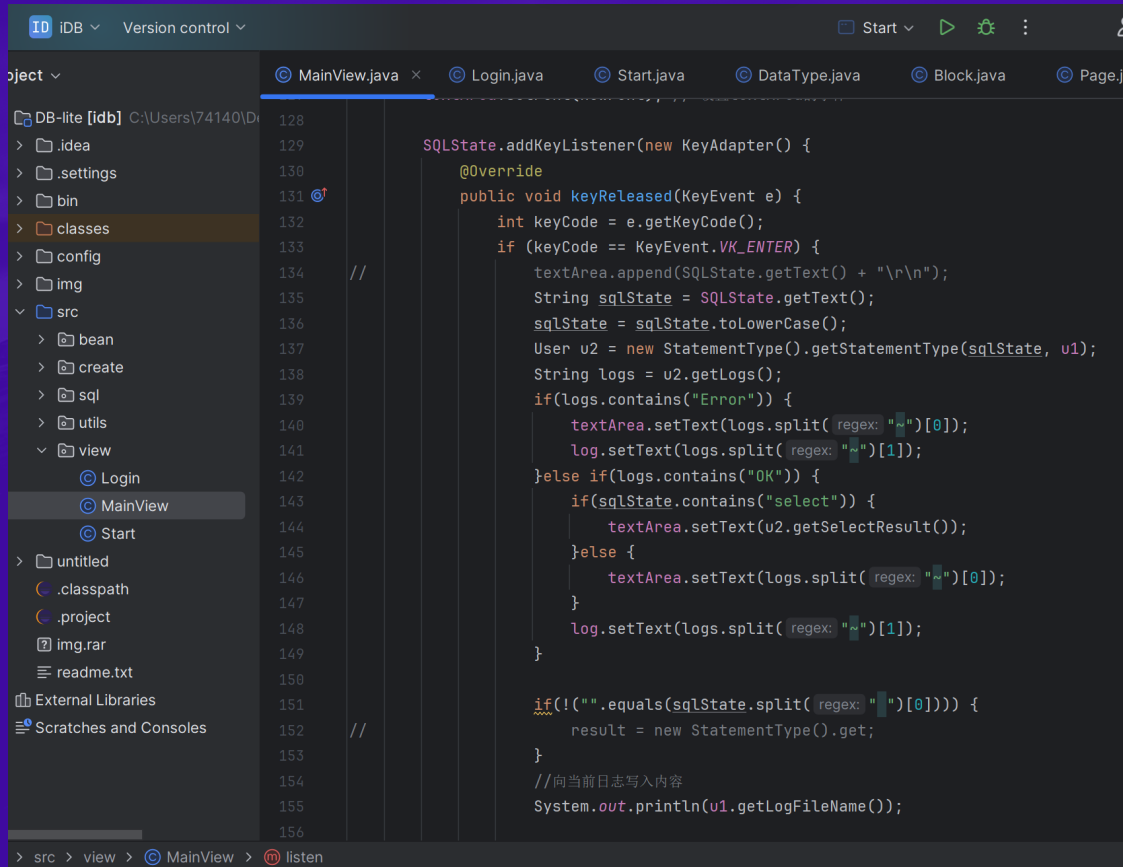
20240116cd7a23e45f03.log	2024-01-16 11:03
202401169fac5fccbe21.log	2024-01-16 11:26
2024011618a4d9685ec4.log	2024-01-16 10:43

```
202401169fac5fccbe21.log
文件 编辑 查看
| AOK~Logs:root(管理员) 权限-all
>>use database 999 success
kError: 1064 - You have an error in your SQL syntax~logs:root(管理员)权限-all
>>operation failure
=OK~Logs: root(管理员) 权限-all
>>select stu1 success
=OK~Logs: root(管理员) 权限-all
>>select stu1 success
=OK~Logs: root(管理员) 权限-all
>>select stu1 success
AOK~Logs: root(管理员) 权限-all
>>delete table stu1 success
AOK~Logs: root(管理员) 权限-all
>>delete table stu1 success
=OK~Logs: root(管理员) 权限-all
```


项目总体框架



部分代码实现



部分代码实现

```
processFileRW.java
35 String[] split = state.replace( target: "(", replacement: " ").split( regex: " "); // 替换
36
37
38 if ("use".equals(split[0])) {
39     // 进入这个数据库，对表进行增删改查
40     String DBDir = FilePath + "\\\" + split[1]; // 数据库文件目录
41     File dbDirFile = new java.io.File(DBDir);
42     if (!dbDirFile.exists()) { // 不存在就报错
43         String log = "Error:1049 - Unknown database '" + split[1] + "'" + "~Logs:"
44             + " \n>>use database " + split[1] + " failure";
45         u1.setLogs(log);
46         return u1;
47     } else { // 存在 就进入这个数据库并获取数据库目录的文件柄 和 日志目录的文件柄
48         File logDir = new java.io.File( pathname: DBDir + "\\logs");
49         if (!logDir.exists()) {
50             logDir.mkdirs();
51         }
52         // 日志文件名
53         String Uuid = UUID.randomUUID().toString().split( regex: " ")[4];
54         Date day = new Date();
55         SimpleDateFormat df = new SimpleDateFormat( pattern: "yyyyMMdd");
56         SimpleDateFormat df1 = new SimpleDateFormat( pattern: "yyyy-MM-dd HH:mm:ss");
57         String LogName = df.format(day) + Uuid + ".log";
58         String logPath = DBDir + "\\logs";
59         File logFile = new File( pathname: logPath+"\\\"+LogName);
60         try {
61             logFile.createNewFile();
62         } catch (IOException e1) {
63             e1.printStackTrace();
64         }
65     }
66 }
```

```
FileMgr.java
50 }
51
52 //把要写入的内容先通过逻辑块blk装入内存页p,再进行写入
53 5 usages
54 public synchronized void write(Block blk,Page p) {
55     try {
56         f.seek( pos: blk.getBlockID() * blkSize);
57         f.getChannel().write(p.contents());
58     } catch (IOException e) {
59         throw new RuntimeException("不能写入块block(blk)+"blk);
60     }
61 }
62
63 //返回追加逻辑块对象(被字节数组初始化过的),失败则报异常
64 1 usage
65 public synchronized Block append(String fileName) {
66     int newBlockID = fileBlockID(fileName);
67     Block blk = new Block(fileName, newBlockID);
68     byte[] b = new byte[blkSize];
69     try {
70         f.seek( pos: blk.getBlockID()*blkSize);
71         f.write(b);
72     } catch (IOException e) {
73         throw new RuntimeException("不能追加blk " + blk);
74     }
75     return blk;
76 }
77
78 //得到文件的逻辑块blk号=块数,因为无论建立日志还是数据文件,
```

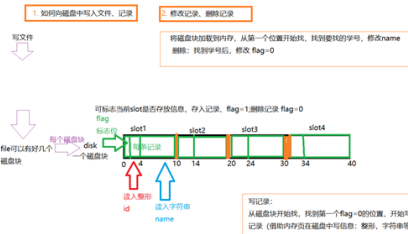

文档整理

语句处理部分 (Sql 包)：根据解析得到的语句类型和条件，执行相应的操作。包括创建数据库和表、删除数据、插入数据、查询数据以及更新数据等操作。对表的模式信息和数据进行记录和更新。 文件读写部分 (工具类)：使用 <code>RandomAccessFile</code> 类进行文件读写，支持随机访问，以复现磁盘操作和缓冲区管理。实现记录的写入、修改和删除，同时记录操作日志。每个数据库对应一个日志文件和一系列表文件。 异常处理：针对不同情况，系统能够报错，包括数据库不存在、数据库已存在、表已存在、未指定使用数据库、SQL 语法错误等情况。错误信息能够提供有用的信息以协助用户识别问题。 效率优化：系统性能的瓶颈在文件 IO 部分，采用 <code>RandomAccessFile</code> 实现随机访问来提高性能。这对于数据库中表文件的读写非常有帮助，特别是在插入和修改数据时能够显著提升效率。 系统采用了图形用户界面，支持多种 SQL 语句操作，具备用户权限管理、异常处理、效率优化等特点，可以满足一个简单的数据库管理系统的基本需求。 3. 系统详细设计 1. 登录系统：分为管理员和普通用户，管理员默认拥有所有权限，对普通用户规定其操作权限，Create 创建数据库操作除外。 用户信息存放在本地文件 Users 里，方便后续代码逻辑使用，信息格式：用户名-密码-是否为管理员 (1 是 0 否) - 对每种操作的权限。 2. 图形化 GUI 界面：用 Swing 构件设计建议的 GUI 图形界面，包括系统登录界面 Login 类和 SQL 语句输入界面 Mainview 类。有效应语句，输入的文本框和操作按钮。顶层容器是 <code>JFrame</code> 框架窗口，中间容器和组件是输入的文本框，功能按钮，标签，背景。给文本框和按钮添加事件监听器，每次登录就是一个 User 对象，方便维护后续所有代码操作。 3. 语句分析： (1) 语句种类判断：从输入文本框里读取内容，先用空格分割成字符串数组 <code>split</code>，根据读入的第一个单词判断是哪种植语句类型。一共有 <code>Create Use Insert Select Delete Update</code> 5 种植语句，然后判断当前用户是否拥有操作的权限，如果有则进入到相应的语句处理部分，没有就报错。 (2) where 条件子句：支持等值比较、不等值比较、大于、小于等条件表达式。数值比较要分为整数比较和浮点数比较，读取符号使用相应的条件判断，对每一个符号都要列率。浮点数比较注意要 > -0.000001 或 < 0.000001 否则会出错。 按照 where 关键字分割得到条件表达式。如果 WHERE 子句中没条件，则返回所有行的下标。因为可以嵌套条件，判断是否存在 WHERE 子句，如果存在，一对 where...from...对应一个筛选条件，根据条件在对应表进行删除更新和查询操作。把符合条件的结果行下标放在一个 <code>ArrayList</code> 里返回，查询失败不返回具体信息。	(3) 数据类型的分析：给每种数据类型 (Int Char String Double Boolean Other) 用数字指代，简化后续代码。设置两个方法实现类型和对应数字的转换。 4. 语句处理部分： 处理前都首先将 SQL 语句中的分号替换为空格，将整个语句转换为小写，然后通过空格分割得到数组 <code>split</code>。 (1) Create use 语句：根据读到的 create 或 use，可以使用库 (同时创建日志)、创建库、创建表。如果是 use 开头，数据库不存在就报错，存在则进入指定数据库，创建日志文件，并记录操作日志；如果是 create database 开头，则创建新的数据库目录 (文件夹)；如果是 create table 开头，调用方法提取字段名、属性和长度，由此创建新的数据表文件，并记录操作日志，同时写入模式表 <code>schema.dat</code> (存储表模式结构)。 <pre>CREATE database <u>USE</u> ;</pre> <pre>CREATE table stu1 (<u>String</u>(9), age int(4), class <u>String</u>(4), grade double(8));</pre> (2) Delete 语句：按照 where 语句条件删除表中信息。本质上是先将表中指定行的 flag 置为 false，解析表名，获取待删除表的信息，根据 where 子句中的条件获取待删除的行索引。打开待删除表文件，将指定行的 flag 置为 false 表示删除。<pre>DELETE FROM stu1 WHERE <u>grade</u> = '张11';</pre> (3) Insert 语句：一次只能插入一条数据，无数据则默认为 0 或空字符串。插入完成后关闭 <code>RandomAccessFile</code>，通过解析插入语句，获取表名、字段名和对应的值。 判断数据库是否存在该表，若存在则读取表的模式信息。遍历表的字段，根据字段类型将值插入到相应的位置。如果字段在插入语句中没有对应的值，则默认插入该字段的默认值。<pre>INSERT INTO 表名称(字段) values(数据); INSERT INTO student(id,NAME) VALUES(1,'学生1');</pre><pre>INSERT INTO 表名称(字段1, 字段2, 字段3) values(数据1, 数据2, 数据3); (数据1, 数据2, 数据3);</pre> (4) Select 语句： 用 where 和 from 分离，获取查询条件和表名。获取满足条件的行号。如果表文件长度为 0 或者查询下标为空，返回相应的错误信息。据查询语句中的字段部分，获取要查询的字段名。如果字段为 *，则查询所有字段；如果为具体字段名，则查询指定字段。通过表的数据结构和行号，将满足条件的字段数据输出为字符串。如果用户权限为非管理员，则只输出 flag 为 true 的行数据，否则全部输出。输出的数据包括表头和查询结果。<pre>select 字段1,字段2,..... 表达式 from 表名 [where 条件] select * from 表名 where 条件 (Qs; flag == true) 若为管理员，全显示</pre> (5) Update 语句：按照条件更新表某行某列数据。通过模式表信息确定字段的类型和位置。打开表文件，根据字段类型和偏移量，将更新的值写入相应位置。更新操作只在满足条件的行中进行。<pre>UPDATE stu1 SET class="2000" WHERE grade > 75;</pre> 5. 文件读写部分：
--	--

使用 java 的 `RandomAccessFile` 类。能够复现磁盘操作、缓冲区管理内容。

实现向磁盘写入文件和记录：file 可以有多个磁盘块，对于每个磁盘块 disk，用标志位 flag 标志当前 slot 是否存在信息。存入记录令 flag=1，删除记录令 flag=0。每条记录依次写入，从磁盘块第一个位置开始找到 flag=0 的位置写入，借助内存预写信息 (如整形 id，字符串 name)。

修改和删除同理，将磁盘块加载到内存，从第一个位置开始找，找到要修改的内容，修改对应字段。删除则是令 flag=0 即可。



(1) 日志文件记录：每次操作的同时创建并写入日志 (包括操作概述，用户，以及操作成功与否)，每次写入后都要关闭日志文件。每次登录都维护一个 log 文件，记录用户，操作，成功与否的信息。日志文件在每个数据库对应文件夹下的 logs 文件夹里，命名格式："xxxxMMdd" + "Uuid" + ".log" eg. 2024011566dd423cfa91.log。使用 `FileMngt` 管理文件，Page 表示内存页，Block 表示块。(这三个属于自己实现的工具类)

(2) 表模式结构文件和表内容的记录：实现原理类似日志文件，具体逻辑在语句处理部分已经涉及到了。在数据库对应文件夹下，分别是 `schema.dat` 和表名 `.dat` 文件。每次操作都会对其中的内容进行同步更新。

6. 异常处理：针对不同情况报错 (语法错误+执行失败)

可能涉及到的报错内容：

Error: Unknown database 数据库不存在

Error: Can't create database 数据库已经存在，无法创建

Error: Table already exists 表已经存在，无法创建

Error: Unselected database 还未指定使用具体的数据库

Error: You have an error in your SQL syntax sql 语句有误

Errors: Table doesn't exist INSERT failure 表不存在，插入失败

Errors: Unknown error occurred INSERT IOTG table failure 未知插入错误

Error: file is empty 文件为空

Error: Unknowable error 位置错误

Error: Access denied for user 访问被拒绝

7. 效率优化：系统的性能瓶颈在文件 IO 部分。文件读写我们一开始选择使用 `BufferedWriter` 和 `BufferedReader`，但是它并不支持直接的随机访问，每次写入都是从文件的当前位置顺序写入。导致在插入和修改数据时需要遍历较长时间。而 `RandomAccessFile` 提供了对文件的随机访问，能够直接跳转到文件的任意位置读写数据。

适用于需要定位到文件中某个位置进行读写的情况，符合数据库中表文件的性质。

8. 存在的问题：

(1) 我们系统当前只支持单条数据的插入和修改，如果后续进行大规模数据处理时，直接的文件 IO 操作可能不够高效，考虑采用缓存策略或异步 IO 以提高性能。

(2) 当前使用固定大小的块进行数据存储，可能会导致存储空间的浪费。如果后续数据规模扩大，需要使用更灵活的存储结构，如动态大小的块或者页来进一步提高存储效率。

(3) 目前的系统只支持单个用户同时访问，如果多用户并发访问，需要考虑并发控制机制。在真实的数据库系统中，需要处理并发访问和事务的问题，以确保数据的一致性和隔离性。

(4) 异常处理机制，我们的系统已经支持一些特定的异常机制，但只是简单的报错，没有进一步的处理。

(5) 当前系统缺乏索引和查询优化的机制，对于大型数据集的查询可能效率较低。后续考虑引入索引结构和查询优化策略，从而提高查询性能。

4. 系统实现和测试

1. 图形化 GUI 界面

Login.java

实现了一个简单的登录窗口，用于用户登录到数据库管理系统 (DBMS-lite)。通过读取文件验证用户信息，根据权限跳转到相应的主界面。

继承 `JFrame` 类：

Login 类继承自 `JFrame`，表示登录窗口是一个顶层窗口。

图形组件的初始化：

使用 `BufferedImage` 加载图片资源，并通过 `ImageIO.read` 方法读取图片文件。

使用 `JLabel` 显示欢迎消息和用户名、密码标签。

使用 `JTextField` 和 `JPasswordField` 分别用于输入用户名和密码。

使用 `JButton` 实现登录和注册按钮。

字体设置：

使用 Font 对象设置不同字体风格，以美化界面。

布局管理：

采用自定义的空布局 (绝对定位布局)，通过 `setBounds` 方法设置组件的位置和大小。

事件监听：

使用 `addActionListener` 方法为登录和注册按钮添加匿名内部类的事件监听器。

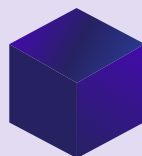


项目回顾

在这个项目中，我们成功地创建了一个DBMS系统，实现了SQL语句的编译和处理，并通过添加功能和模块化函数来完善系统。提供了多个工具类来处理数据库操作。还通过前端设计来给用户好的交互体验。

项目总结

Project Summary



经验教训

在这个项目中，我们学到了许多有关模块类的编写和它们之间的相互连接的经验。我们也了解了在每个模块中可能会遇到的代码编写问题。这些经验也将有助于我们更好地应对类似的挑战，并提高未来项目的效率和质量。